# Task 3 Part B: Testing POST /api/v1/payments

## 3.1 Highest-risk scenarios

What makes this a money endpoint, not ordinary CRUD:

- **It can't simply be undone:** once money leaves, it's gone.
- **Retries can pay twice.** See 3.2.
- **Rounding moves real money:** amounts must be exact.
- **Every call is checked:** each payment is limited by balance, limits and who owns the account.

**The amount object: value, currency, precision**

| Scenario | Expected |
|---|---|
| value is 0, negative, missing or null | 400. Nothing debited. |
| value sent as a string ("1500.75") or in scientific notation (1.50075e3) | 400. The API shouldn't guess what was meant. |
| More decimals than the currency allows: SAR 1500.755, JPY 100.5, KWD 1.2345 | 400. Never rounded silently. SAR has 2 decimals, JPY 0, KWD 3. |
| 1500.75 kept exactly, with no 1500.7499999 in the debit or the fee | Amounts are stored as exact minor units |
| A very large value (99999999.99) | 422 or 400, with no crash and no overflow |
| currency is lowercase, unknown (XYZ), 4 letters (SARR) or missing | 400. Never cut down or guessed |
| currency differs from the source account's currency, and there's no fxQuoteId | 400 or 422. A quote is required. |
| fxQuoteId is expired, belongs to another customer, was already used, or doesn't match the amount or currency | 400 or 409. Never re-quoted silently at a different rate. |

**Limits (422) and insufficient funds (402)**

| Scenario | Expected |
|---|---|
| Exactly at the limit, then 0.01 over | 201, then 422 |
| Daily total built up over several payments and channels (X-Channel) | 422 once the total crosses the limit. Changing X-Channel doesn't bypass it. |
| Two payments sent at the same moment that together go over the limit or the balance | Exactly one succeeds |
| Balance equal to amount + fee, then 0.01 short | 201, then 402 |
| The amount fits the balance but amount + fee doesn't | 402. The fee counts in the balance check. |
| After any 402 or 422 | Nothing debited, no fee, nothing counted toward the limit, and no payment left in PENDING |
| Over the limit and short of money at the same time | The same error every time, in an agreed order |

**Who can move what**

| Scenario | Expected |
|---|---|
| sourceAccountId belongs to another customer | Rejected, revealing nothing about that account |
| beneficiaryId isn't registered to this customer | Rejected |
| Authorization is missing, expired or invalid | 401 |

| | |
|---|---|
| purposeCode is missing or invalid; note is very long or contains script | 400, or stored safely as plain text |

## 3.2 Idempotency-Key

**What it's for:** it lets the client retry a payment safely. The same key always means the same payment, so a retry never causes a second debit.

**How I'd test it**

| Test | Expected |
|---|---|
| Send the same request twice with the same key | Same paymentId and reference both times, and one debit |
| Send the same request twice at the exact same moment, with the same key | One payment. The other call gets the same result, or 409 "still in progress". |
| Same key, but a different amount or beneficiary | 409. Nothing new is paid. |
| Different key, same body | A second, separate payment. A new key means "this is a new payment". |
| No key | 400. Payments shouldn't be accepted without one. |
| Another customer uses the same key | Handled separately. They never see the first customer's payment. |
| Retry after the original got 400, 402 or 422 | The same stored answer. A genuinely new attempt needs a new key. |
| Retry after the key has expired (for example, after 24 hours) | Agreed and documented behaviour |

**Retry after a network timeout:** the client doesn't know whether the payment happened. It must retry with the same key and the same body.

- If the first call went through, the server returns the original result, with no second debit.
- If it didn't go through, the server processes it now, once.
- To test it: make the server reply after the client's timeout, retry with the same key, then check the bank's records show exactly one debit.

**Retry after a 503:** a 503 should mean "nothing was done", so a retry with the same key is safe and pays once.

- I'd test the dangerous case. A downstream system fails after the debit has already happened. The API must not return 503 if money has moved. If it does, the retry with the same key must return the original payment, not debit again.
- The client should wait before retrying (using a Retry-After header) rather than hammering the API.

## 3.3 How I'd verify the money actually moved

A 201 only means the API said yes. I'd check the bank's records (the ledger) and every place the payment should show up:

- **Source account:** debited the amount plus the fee exactly once, as separate entries. The balance before, minus (amount + fee), equals the balance after.
- **Receiving side:** for the same bank, the beneficiary is credited the amount. For another bank, an outgoing payment is created for the payment network, with the same amount and reference.
- **Reference:** the entries carry the paymentId and reference, and the reference is unique.
- **Read back:** fetching the payment, the transaction history and the statement all show the same amount, fee, status and reference as the 201.
- **PENDING payments:** follow them to their final status. If one ends as FAILED, the amount and fee are refunded in full and the limit usage is given back.

- **Fee and FX:** the fee lands in the bank's fee account. For FX payments, the rate used matches the fxQuoteId, and the converted amount equals value × rate.
- **Limit usage:** today's limit usage went up by the amount.
- **Failures:** after a 4xx or 503, there's no ledger entry, no fee and no hold left on the account.